

GUÍA COMPLETA

RAG Local con n8n

Retrieval-Augmented Generation desde cero hasta producción

Docker · PostgreSQL · pgvector · n8n · Ollama · OpenAI

6

Fases completas

30+

Comandos paso a paso

2

Sistemas operativos

100%

Ejecución local

¿Qué aprenderás con esta guía?

- Instalar Docker Desktop en macOS y Windows desde cero
- Instalar Node.js y n8n como motor de automatización
- Desplegar PostgreSQL con la extensión pgvector para búsqueda semántica
- Configurar Ollama para usar modelos de IA 100% en local (sin internet)
- Construir el flujo de ingesta incremental de documentos en n8n
- Generar embeddings y almacenarlos en la base de datos vectorial
- Ejecutar consultas RAG y obtener respuestas contextualizadas
- Diagnosticar y resolver los errores más comunes

Versión 2.0 · Junio 2025 · Compatible: n8n $\geq$ 1.40 · Docker Desktop $\geq$ 4.x · Node.js $\geq$ 18

Índice de Contenidos

FASE 0 Requisitos Previos y Descarga de Herramientas

- 0.1 Requisitos de hardware y software
- 0.2 Instalar Docker Desktop en macOS
- 0.3 Instalar Docker Desktop en Windows
- 0.4 Instalar Node.js
- 0.5 Instalar n8n globalmente
- 0.6 (Opcional) Instalar Ollama para modelos locales

FASE 1 Infraestructura de Datos (Docker + pgvector + pgAdmin)

- 1.1 Lanzar PostgreSQL con pgvector
- 1.2 Lanzar pgAdmin 4
- 1.3 Conectar pgAdmin a la base de datos
- 1.4 Activar extensión e inicializar el esquema SQL

FASE 2 Configuración del Entorno n8n

- 2.1 Por qué n8n necesita permisos especiales
- 2.2 Detener instancias en ejecución
- 2.3 Lanzar n8n con acceso al sistema de archivos

FASE 3 Construcción del Flujo de Ingesta RAG en n8n

- 3.1 Arquitectura del flujo
- 3.2 Crear la carpeta de ingesta
- 3.3 Nodos: Schedule Trigger → Read Binary Files
- 3.4 Nodos: Data Loader + Text Splitter + Embeddings
- 3.5 Nodo: Postgres Vector Store
- 3.6 Nodo: Execute Command (purga)
- 3.7 Activar el flujo

FASE 4 Flujo de Consulta RAG (Chat con tus Documentos)

- 4.1 Arquitectura del flujo de consulta
- 4.2 Nodos: Chat Trigger + AI Agent + LLM
- 4.3 Nodo: Postgres Vector Store (modo Retrieve)

FASE 5 Verificación y Pruebas

- 5.1 Verificar datos indexados en pgAdmin
- 5.2 Pruebas del flujo de consulta

FASE 6 Solución de Problemas y Referencia Técnica

- 6.1 Tabla de errores frecuentes y soluciones
- 6.2 Diferencias macOS vs Windows

FASE 0

Requisitos Previos y Descarga de Herramientas

Todo lo que necesitas instalar antes de empezar

0.1 Requisitos de Hardware y Software

El sistema RAG ejecuta simultáneamente PostgreSQL, pgAdmin y n8n, por lo que es importante cumplir los requisitos mínimos para una experiencia fluida.

Componente	Mínimo	Recomendado	Notas
RAM	8 GB	16 GB	Con Ollama local se recomiendan 16 GB
CPU	4 núcleos	8 núcleos	Apple Silicon o AMD Ryzen para Ollama
Disco	20 GB libres	50 GB libres	Imágenes Docker + modelos Ollama
Sistema Op.	macOS 12 / Win 10 22H2	macOS 14 / Win 11	Soporte nativo de contenedores
Internet	Solo durante la instalación	10 Mbps+	Para descargar imágenes y modelos

0.2 Instalar Docker Desktop en macOS

Docker es la plataforma de contenedores que usaremos para ejecutar PostgreSQL y pgAdmin de forma aislada. Docker Desktop incluye el motor Docker Engine y una interfaz gráfica.

1

Descargar Docker Desktop para Mac

Visita la página oficial y descarga el instalador:

Descarga oficial

<https://www.docker.com/products/docker-desktop>

Elige la versión correcta para tu Mac:

- **Mac con Apple Silicon (M1/M2/M3/M4)** → selecciona "Apple Chip"
- **Mac con Intel** → selecciona "Intel Chip"

Si no sabes tu procesador: menú Apple → **Acerca de este Mac**.

2**Instalar la aplicación**

Haz doble clic en el archivo Docker.dmg descargado. Arrastra el icono de Docker a la carpeta Aplicaciones.

Al abrir Docker por primera vez pedirá tu contraseña de administrador. Acepta y espera a que el icono de la ballena en la barra de menú quede estático y muestre el mensaje "Docker Desktop is running".

3**Verificar la instalación en Terminal**

Abre Terminal (Cmd+Espacio → escribe Terminal) y ejecuta:

```
docker --version
docker run hello-world
```

Deberías ver el número de versión y un mensaje de bienvenida de Docker.

0.3 Instalar Docker Desktop en Windows

En Windows, Docker Desktop requiere el Subsistema de Windows para Linux (WSL 2). El instalador lo activa automáticamente, pero puede necesitar un reinicio.

1**Habilitar WSL 2**

Abre **PowerShell como Administrador** (clic derecho → "Ejecutar como administrador"):

```
wsl --install
```

Reinicia el equipo si se te solicita.

2**Descargar Docker Desktop para Windows**

Visita la misma página y descarga el instalador .exe para Windows:

Descarga oficial

<https://www.docker.com/products/docker-desktop>

Ejecuta el instalador. Asegúrate de que la opción **"Use WSL 2 instead of Hyper-V"** esté marcada. Reinicia cuando se indique.

3**Verificar en PowerShell**

Abre PowerShell y ejecuta:

```
docker --version
docker run hello-world
```

0.4 Instalar Node.js

n8n es una aplicación Node.js. Se recomienda la versión LTS (Long Term Support). La versión mínima requerida por n8n es **Node.js 18**.

1

Descargar Node.js LTS

Visita la web oficial y descarga la versión **LTS** (recomendada para la mayoría):

Descarga oficial

<https://nodejs.org/es>

- **macOS**: descarga el .pkg y sigue el asistente
- **Windows**: descarga el .msi y sigue el asistente

2

Verificar en terminal

En Terminal (macOS) o PowerShell (Windows):

```
node --version
npm --version
```

Deberías ver algo similar a v20.x.x y 10.x.x respectivamente.

0.5 Instalar n8n Globalmente

n8n se instala como paquete global de Node.js con npm. Estará disponible como comando desde cualquier directorio.

1

Instalar con npm

macOS (Terminal) o Windows (PowerShell como Administrador):

```
npm install -g n8n
```

La instalación puede tardar varios minutos. Cuando termine verás el cursor de comandos sin errores en rojo.

2

Verificar la instalación

```
n8n --version
```

Verás el número de versión instalado, por ejemplo: 1.45.0

0.6 (Opcional) Instalar Ollama para Modelos Locales

Ollama permite ejecutar modelos de IA completamente en local, sin enviar datos a servicios externos. Ideal para documentos confidenciales o cuando prefieres no depender de APIs de pago. Si usarás OpenAI, puedes saltar esta sección.

1

Descargar e instalar Ollama

Visita la web oficial y descarga el instalador para tu sistema:

Descarga oficial

<https://ollama.com/download>

- **macOS:** descarga el .dmg y arrastra Ollama a Aplicaciones
- **Windows:** descarga el .exe y ejecútalo

Ollama se ejecuta en segundo plano, accesible en el puerto 11434.

2

Descargar los modelos necesarios

Abre Terminal o PowerShell y ejecuta estos comandos:

```
# Modelo de embeddings (768 dimensiones, ~270 MB)
ollama pull nomic-embed-text
# Modelo de lenguaje para respuestas (~4 GB, opcional)
ollama pull llama3.2
```

Verifica qué modelos tienes con: `ollama list`

3

Verificar que Ollama funciona

Abre tu navegador en esta dirección:

Panel de Ollama

<http://localhost:11434>

Deberías ver el texto "Ollama is running".

■■ IMPORTANTE: Dimensiones de embeddings

Si usas OpenAI (text-embedding-3-small) → la columna debe ser VECTOR(1536)

Si usas Ollama (nomic-embed-text) → la columna debe ser VECTOR(768)

Debes decidir cuál usar ANTES de crear la tabla. Cambiar después requiere recrearla y reindexar todos los documentos.

FASE 1

Infraestructura de Datos

Docker + PostgreSQL + pgvector + pgAdmin

Levantaremos dos contenedores Docker: uno con PostgreSQL 16 + pgvector (búsqueda vectorial nativa) y otro con pgAdmin 4 (interfaz web de administración). Al usar Docker no instalamos nada directamente en la máquina y el entorno es idéntico en cualquier sistema operativo.

1.1 Lanzar PostgreSQL con pgvector

Abre Terminal (macOS) o PowerShell (Windows) y ejecuta:

```
docker run --name postgres-rag \  
-e POSTGRES_PASSWORD=mi_contra_secreta \  
-p 5432:5432 \  
-d pgvector/pgvector:pg16
```

Parámetro	Valor	Significado
--name	postgres-rag	Nombre del contenedor
-e POSTGRES_PASSWORD	mi_contra_secreta	Contraseña del usuario postgres
-p 5432:5432	host:contenedor	Mapea el puerto al localhost de tu máquina
-d	flag	Ejecuta en segundo plano (detached)
pgvector/pgvector:pg16	imagen Docker	PostgreSQL 16 + extensión pgvector preinstalada

Verifica que está corriendo con:

```
docker ps
```

Deberías ver "postgres-rag" en la lista con estado "Up".

1.2 Lanzar pgAdmin 4

Ejecuta este segundo contenedor para la interfaz gráfica:

```
docker run --name pgadmin-rag \
-e PGADMIN_DEFAULT_EMAIL=admin@admin.com \
-e PGADMIN_DEFAULT_PASSWORD=1234 \
-p 8080:80 \
-d dpage/pgadmin4
```

Espera unos 15–20 segundos y luego abre tu navegador en:

pgAdmin 4 (interfaz web) <http://localhost:8080>

Credenciales de acceso: Email: admin@admin.com | Contraseña: 1234

1.3 Conectar pgAdmin a PostgreSQL

Al entrar en pgAdmin por primera vez, registra el servidor de PostgreSQL. Sigue estos pasos:

- En el panel izquierdo: clic derecho sobre "Servers" → "Register" → "Server..."
- Pestaña **General**: escribe el nombre RAG-Local
- Pestaña **Connection**: introduce los siguientes parámetros:

Campo	Valor que debes escribir	Por qué
Host name/address	<code>host.docker.internal</code>	Nombre especial que resuelve al localhost de la máquina desde dentro del contenedor
Port	<code>5432</code>	Puerto estándar de PostgreSQL
Maintenance database	<code>postgres</code>	Base de datos por defecto para la conexión inicial
Username	<code>postgres</code>	Usuario administrador de PostgreSQL
Password	<code>mi_contra_secreta</code>	La contraseña que pusiste en el paso 1.1

Haz clic en "Save". El servidor aparecerá conectado en el panel izquierdo.

1.4 Activar la Extensión e Inicializar el Esquema

Crearemos la base de datos del RAG, habilitaremos pgvector dentro de ella y crearemos la tabla que almacenará los fragmentos de texto con sus embeddings.

1

Crear la base de datos rag_db

En pgAdmin: clic derecho sobre "Databases" → "Create" → "Database..."

En el campo Database escribe: rag_db → haz clic en "Save".

2

Abrir el Query Tool

Selecciona rag_db en el panel izquierdo. Ve a Tools → Query Tool (o el icono de rayo en la barra superior).

3

Ejecutar el SQL de inicialización

Copia y pega el siguiente SQL en el editor y presiona F5 o el botón "Execute":

```
-- 1. Habilitar la extensión de vectores
CREATE EXTENSION IF NOT EXISTS vector;
-- 2. Crear la tabla de documentos
-- Usa VECTOR(1536) para OpenAI o VECTOR(768) para Ollama
CREATE TABLE IF NOT EXISTS documents (
  id          SERIAL PRIMARY KEY,
  content     TEXT,
  metadata    JSONB,
  embedding   VECTOR(1536)
);
-- 3. Índice para búsquedas por similitud (acelera las consultas)
CREATE INDEX IF NOT EXISTS documents_embedding_idx
  ON documents USING ivfflat (embedding vector_cosine_ops)
  WITH (lists = 100);
```

Si la ejecución es correcta verás: "Query returned successfully" en el panel inferior.

■ ■ Sobre el índice ivfflat

El índice ivfflat acelera la búsqueda de similitud vectorial. Sin él, cada consulta compara el vector de búsqueda contra TODOS los documentos (lento con miles de registros). Con el índice, la búsqueda se limita a candidatos relevantes.

lists = 100 es adecuado para colecciones de hasta ~1 millón de documentos.

FASE 2

Configuración del Entorno n8n

Lanzamiento con acceso al sistema de archivos local

Por defecto n8n bloquea el acceso al sistema de archivos por seguridad. Para que el flujo RAG lea documentos de una carpeta local, debemos desactivar esta restricción con una variable de entorno. Esto requiere detener n8n y relanzarlo con los parámetros correctos.

2.1 Por Qué n8n Necesita Permisos Especiales

La variable **N8N_BLOCK_ENV_ACCESS_IN_NODE** controla si n8n puede acceder al sistema de archivos del host. Por defecto está en "true" (bloqueado). Necesitamos establecerla en "false" para que el nodo "Read Binary Files" pueda leer nuestra carpeta de entrada de documentos.

2.2 Detener Instancias en Ejecución

Antes de relanzar n8n, asegúrate de que no hay ninguna instancia corriendo en el puerto 5678.

En macOS (Terminal):

```
# Busca y finaliza el proceso que ocupa el puerto 5678
kill -9 $(lsof -t -i:5678)
# Si no hay ninguno corriendo, el comando dará un error inofensivo
```

En Windows (PowerShell como Administrador):

```
# Detiene todos los procesos de Node.js (que incluye n8n)
Stop-Process -Name "node" -Force
# Alternativa: buscar el proceso específico en el puerto 5678
$pid = (Get-NetTCPConnection -LocalPort 5678 -ErrorAction SilentlyContinue).OwningProcess
if ($pid) { Stop-Process -Id $pid -Force }
```

2.3 Lanzar n8n con Acceso al Sistema de Archivos

Mantén esta terminal abierta mientras trabajas: cerrarla detendrá n8n.

En macOS (Bash/Zsh):

```
export N8N_BLOCK_ENV_ACCESS_IN_NODE=false && n8n start
```

En Windows (PowerShell): — dos comandos separados

```
# 1. Establecer la variable de entorno
$env:N8N_BLOCK_ENV_ACCESS_IN_NODE="false"
# 2. Iniciar n8n
n8n start
```

1

Acceder a la interfaz de n8n

Una vez que n8n esté corriendo, abre tu navegador en:

Interfaz n8n

<http://localhost:5678>

La primera vez te pedirá crear una cuenta de administrador local (email y contraseña ficticios, es solo local). Después verás el canvas de n8n.

■ ■ Consejo: script de arranque rápido

Para no escribir el comando cada vez, crea un script:

macOS: guarda el comando en start-n8n.sh y dale permisos con `chmod +x start-n8n.sh`

Windows: guarda los dos comandos en start-n8n.ps1 y ejecútalo con PowerShell.

FASE 3

Construcción del Flujo de Ingesta RAG en n8n

Monitorización automática → Embeddings → Base de datos vectorial

Este flujo se ejecuta automáticamente según una programación. Lee los archivos de una carpeta, los divide en fragmentos, genera embeddings y los almacena en PostgreSQL. Al final elimina los archivos procesados para evitar duplicados.

3.1 Arquitectura del Flujo de Ingesta

#	Nodo	Tipo	Función
1	Schedule Trigger	Trigger	Activa el flujo cada X minutos
2	Read Binary Files	Input	Lee todos los archivos de la carpeta de entrada
3	Default Data Loader	AI · Document	Convierte el binario en documento procesable
4	Recursive Text Splitter	AI · Splitter	Divide el texto en fragmentos con solapamiento
5	Embeddings (OpenAI/Ollama)	AI · Embeddings	Genera vectores numéricos para cada fragmento
6	Postgres Vector Store	AI · Vector Store	Almacena fragmentos y vectores en PostgreSQL
7	Execute Command	Output	Elimina los archivos ya procesados

3.2 Preparar la Carpeta de Ingesta

Crea la carpeta donde depositarás los documentos a indexar:

macOS:

```
mkdir -p ~/n8n_rag_input
```

Windows (PowerShell):

```
New-Item -ItemType Directory -Path "C:\n8n_rag_input" -Force
```

3.3 Crear el Flujo en n8n

Accede a n8n en <http://localhost:5678>, haz clic en "**New Workflow**" y comienza a añadir nodos con el botón "+".

1

Nodo: Schedule Trigger

Cómo añadirlo: "+" → busca "Schedule Trigger" → selecciónalo.

Configuración:

- Trigger Interval: **Minutes**
- Minutes Between Triggers: **5** (se ejecutará cada 5 minutos)

Este nodo es el punto de inicio. Se activará automáticamente sin intervención manual.

2

Nodo: Read Binary Files

Cómo añadirlo: "+" → "Read Binary Files".

Configuración del campo "File Path":

- **macOS:** /Users/tu_usuario/n8n_rag_input/*
- **Windows:** C:\n8n_rag_input*

El asterisco (*) es un comodín que lee todos los archivos de la carpeta. Sustituye "tu_usuario" por tu nombre de usuario real.

■ ■ Si n8n no puede acceder a la ruta...

Asegúrate de haber lanzado n8n con `N8N_BLOCK_ENV_ACCESS_IN_NODE=false` (Fase 2).

Si relanzaste n8n sin ese flag, detén el proceso y vuelve a iniciarlo correctamente.

3

Nodo: Postgres Vector Store (modo Insert Documents)

"+" → busca "Postgres PGVector Store" → selecciona el modo "**Insert Documents**".

Este nodo tiene tres **entradas auxiliares** que debes conectar desde sus propios conectores:

Configuración de credenciales de la base de datos:

Campo	Valor
Host	<code>localhost</code> (o <code>127.0.0.1</code>)
Port	<code>5432</code>
Database	<code>rag_db</code>
User	<code>postgres</code>
Password	<code>mi_contra_secreta</code>
Table Name	<code>documents</code>

Campo	Valor
Query Name (columna vectorial)	<code>embedding</code>

4

Sub-nodo: Default Data Loader (entrada "Document")

En el nodo Postgres Vector Store, haz clic en el conector "Document" → busca "Default Data Loader".

Configuración:

- Binary Data Key: **data** (identificador estándar del binario leído)

Este nodo convierte el archivo binario en un documento procesable por el modelo de embeddings.

5

Sub-nodo: Recursive Character Text Splitter (entrada "Text Splitter")

En el nodo Postgres Vector Store, haz clic en el conector "Text Splitter" → "Recursive Character Text Splitter".

Configuración:

- Chunk Size: **1000** caracteres
- Chunk Overlap: **200** caracteres

El overlap garantiza que el contexto no se pierda en los bordes entre fragmentos.

■ ■ ¿Por qué 1000 / 200?

Son valores de partida para documentos en prosa estándar. Para documentos técnicos con párrafos cortos prueba 500/100. Para documentos muy largos con mucho contexto, puedes subir a 2000/400.

6

Sub-nodo: Embeddings (entrada "Embeddings")

Haz clic en el conector "Embeddings" del nodo Postgres Vector Store:

Opción A — OpenAI Embeddings (requiere cuenta de pago):

- Busca "Embeddings OpenAI" → Model: **text-embedding-3-small**
- Crea una credencial con tu API Key de OpenAI

Opción B — Ollama Embeddings (100% local, gratis):

- Busca "Embeddings Ollama"
- Base URL: `http://localhost:11434`
- Model: **nomic-embed-text**

■ ■ Dimensión de vectores: debe coincidir con la tabla

OpenAI text-embedding-3-small → VECTOR(1536)

Ollama nomic-embed-text → VECTOR(768)

Si usaste un proveedor y quieres cambiarlo, debes recrear la tabla y reindexar.

7

Nodo: Execute Command (purga de archivos procesados)

Conecta la **salida de éxito** del nodo Postgres Vector Store a un nuevo nodo "Execute Command".

Comando para macOS:

```
rm -f /Users/tu_usuario/n8n_rag_input/*
```

Comando para Windows:

```
Remove-Item -Path "C:\n8n_rag_input\*" -Force
```

Este nodo evita que los archivos ya procesados vuelvan a indexarse en la siguiente ejecución del flujo.

8

Activar el flujo

Una vez configurados todos los nodos, haz clic en el toggle "**Active**" en la esquina superior derecha del canvas.

Para probarlo manualmente antes de activarlo, haz clic en "**Test Workflow**". Asegúrate de tener al menos un archivo en la carpeta de ingesta.

FASE 4

Flujo de Consulta RAG

Chat con tus documentos indexados

Una vez indexados los documentos, este segundo flujo recibe preguntas del usuario, busca los fragmentos más relevantes en PostgreSQL y usa un LLM para generar una respuesta contextualizada con la información de tus documentos.

4.1 Arquitectura del Flujo de Consulta

#	Nodo	Tipo	Función
1	Chat Trigger	Trigger	Recibe la pregunta del usuario
2	AI Agent	AI · Agent	Orquesta la búsqueda y genera la respuesta
3	LLM (OpenAI o Ollama)	AI · LLM	Modelo de lenguaje para generar la respuesta
4	Postgres Vector Store	AI · Vector Store	Recupera fragmentos similares a la pregunta
5	Embeddings (mismo que ingesta)	AI · Embeddings	Convierte la pregunta a vector para buscar

4.2 Construir el Flujo de Consulta

Crea un nuevo workflow en n8n y añade los siguientes nodos:

1

Nodo: Chat Trigger

"+" → "Chat Trigger".

Este nodo crea una interfaz de chat integrada en n8n y expone un webhook URL para integrar el chat con otras aplicaciones. No requiere configuración adicional.

Al hacer clic en el botón "Chat" del canvas se abrirá la ventana de chat.

2

Nodo: AI Agent

"+" → "AI Agent".

System Message recomendado (pégalo en el campo correspondiente):

"Eres un asistente experto. Usa ÚNICAMENTE la información del contexto proporcionado para responder. Si no encuentras la respuesta en el contexto, indícalo claramente. No inventes datos."

Al AI Agent debes conectar dos sub-nodos:

3

Sub-nodo LLM: OpenAI Chat Model o Ollama

Opción A — OpenAI (conector "Model"):

- Busca "OpenAI Chat Model"
- Model: **gpt-4o-mini** (balance coste/calidad) o gpt-4o

Opción B — Ollama (100% local, conector "Model"):

- Busca "Ollama Chat Model"
- Base URL: `http://localhost:11434`
- Model: **llama3.2** (debe estar descargado con `ollama pull llama3.2`)

4

Sub-nodo: Postgres Vector Store en modo Retrieve

En el AI Agent, haz clic en el conector "**Tool**" → busca "Postgres PGVector Store" → selecciona el modo "**Retrieve Documents (As Tool for AI Agent)**".

Configuración: mismos datos de conexión que en el flujo de ingesta.

- Table Name: **documents**
- Query Name: **embedding**
- Limit: **4** (número de fragmentos más relevantes a recuperar)

Importante: conecta también el mismo tipo de nodo de Embeddings (OpenAI o Ollama) que usaste en la ingesta, ya que la búsqueda necesita vectores en el mismo espacio dimensional.

5

Activar y probar el flujo de consulta

Activa el flujo y haz clic en el botón "**Chat**" del canvas. Escribe una pregunta relacionada con los documentos indexados. Ejemplo:

"¿Cuál es el proceso de instalación descrito en el documento?"

El sistema buscará fragmentos relevantes, los pasará como contexto al LLM y generará una respuesta basada exclusivamente en tu documentación.

FASE 5

Verificación y Pruebas

Confirmar que el sistema RAG funciona correctamente

5.1 Verificar que los Datos se Han Indexado

Después de ejecutar el flujo de ingesta con al menos un documento, verifica en pgAdmin que los embeddings se almacenaron correctamente.

1

Abrir Query Tool en pgAdmin

Abre pgAdmin en:

pgAdmin

<http://localhost:8080>

Selecciona rag_db → Tools → Query Tool

2

Ejecutar consultas de verificación

Copia y ejecuta el siguiente SQL:

```
-- ¿Cuántos fragmentos hay indexados?
SELECT COUNT(*) FROM documents;
-- Vista previa de los primeros 5 fragmentos
SELECT id, LEFT(content, 120) AS preview, metadata
FROM documents LIMIT 5;
-- ¿Todos tienen embedding?
SELECT COUNT(*) FROM documents WHERE embedding IS NOT NULL;
-- Fragmentos por documento fuente
SELECT metadata->>'source' AS fuente, COUNT(*) AS fragmentos
FROM documents
GROUP BY metadata->>'source'
ORDER BY fragmentos DESC;
```

5.2 Pruebas del Flujo de Consulta

Realiza estas pruebas para confirmar que el RAG funciona de extremo a extremo:

- **Prueba 1 — Pregunta directa:** pregunta algo que esté claramente en tus documentos. La respuesta debe parafrasear el contenido relevante.
- **Prueba 2 — Fuera de alcance:** pregunta algo que NO esté en tus documentos. El sistema debe responder que no tiene información, no inventar.

- **Prueba 3 — Síntesis:** pregunta algo que requiera combinar información de varios fragmentos. Evalúa si el contexto se recupera correctamente.
- **Prueba 4 — Idioma y formato:** verifica que las respuestas se generan en el idioma de tus documentos y con el formato esperado.

Solución de Problemas y Referencia Técnica

Errores frecuentes, causas y soluciones

6.1 Tabla de Errores Frecuentes

Error / Síntoma	Causa	Solución
Cannot connect to Docker daemon	Docker Desktop no está corriendo	Abre Docker Desktop y espera a que el icono de la ballena se estabilice
pgAdmin: "Connection refused"	El contenedor de Postgres no está activo	Ejecuta: docker start postgres-rag
n8n: "ENOENT: no such file or directory"	La ruta de la carpeta de ingesta no existe	Crea la carpeta con mkdir o New-Item
n8n no lee los archivos / "Access denied"	N8N_BLOCK_ENV_ACCESS_I N_NODE no fue desactivado	Detén n8n y relánzalo con el flag correcto (Fase 2)
Error de dimensiones vectoriales	El modelo de embeddings no coincide con VECTOR(N) de la tabla	Recrea la tabla con la dimensión correcta: 1536 (OpenAI) o 768 (Ollama)
Ollama: "model not found"	El modelo no fue descargado	Ejecuta: ollama pull nomic-embed-text
Ollama: "connection refused" en puerto 11434	Ollama no está corriendo	Inicia la app Ollama (macOS) o ejecuta ollama serve en terminal
n8n: puerto 5678 ya en uso	Hay una instancia previa de n8n corriendo	Sigue el proceso de la Fase 2 para detener instancias previas
Documentos duplicados en la BD	El nodo Execute Command no ejecutó la purga correctamente	Verifica la ruta del comando rm / Remove-Item y que esté conectado a la salida de éxito
El LLM inventa respuestas	El System Message no restringe al LLM al contexto	Añade instrucciones explícitas: "Usa ÚNICAMENTE el contexto proporcionado"

6.2 Diferencias macOS vs Windows

Acción	macOS (Terminal / Zsh)	Windows (PowerShell)
Instalar Docker	Archivo .dmg → arrastrar a Aplicaciones	Instalador .exe + WSL 2 habilitado

Acción	macOS (Terminal / Zsh)	Windows (PowerShell)
Instalar Node.js	Instalador .pkg o <code>nvm install --lts</code>	Instalador .msi o <code>winget install OpenJS.NodeJS</code>
Crear carpeta de ingesta	<code>mkdir -p ~/n8n_rag_input</code>	<code>New-Item -ItemType Directory -Path "C:\n8n_rag_input"</code>
Liberar puerto 5678	<code>kill -9 \$(lsof -t -i:5678)</code>	<code>Stop-Process -Name "node" -Force</code>
Inyectar variable de entorno	<code>export VAR=false && n8n start</code>	<code>\$env:VAR="false" (comando separado)</code> <code>n8n start</code>
Rutas de archivo	<code>/Users/usuario/n8n_rag_input/*</code>	<code>C:\n8n_rag_input*</code>
Eliminar archivos procesados	<code>rm -f /Users/usuario/n8n_rag_input/*</code>	<code>Remove-Item -Path "C:\n8n_rag_input*" -Force</code>
Host Postgres desde pgAdmin	<code>host.docker.internal</code>	<code>host.docker.internal</code>

Sistema RAG Local Completado con Éxito

Has configurado un sistema completo de RAG que funciona 100% en tu máquina local. Tus documentos nunca salen de tu equipo.